



PROJECT 2 PROPOSAL

PantryPlate

MUHAMMAD IKHWAN BIN WAHID

KOH WE KIAT

ANASTASIA FRANCES FREDERICA

ALEKHYA KODAVATIGANTI

ABINAV SHAJIL

A study of how recommendations evolve as users tune the relative importance of competing objectives.

“What should I cook tonight?”

Harder than existing recommenders treat it as.

Real cooks juggle four constraints simultaneously:

Pantry

What's on hand and available

Nutrition

Calorie and macro alignment

Diet

Vegetarian, gluten-free,
allergens

Taste

What they'd actually enjoy

GAP

Existing recommenders (Spoonacular, MyFitnessPal, Eat This Much) optimize for one constraint at a time. None model the trade-off explicitly.

The trade-off as our research object

We make multi-constraint balancing the unit of study, not a side-effect.

We build a recommender that explicitly models the joint trade-off between four objectives, and study how recommendations evolve as users tune the relative weights.

HEADLINE HYPOTHESIS

Balanced multi-objective ranking produces qualitatively better recommendations than single-objective optimization on a composite usefulness metric.

The slider between objectives isn't a UI feature — it's the empirical finding.

Food.com via Kaggle

Recipes

231,637

Total reviews

1.13M

All users

226,570

PUBLISHED PRE-SPLIT · TRAIN / VALIDATION / TEST (MAJUMDER ET AL. 2019)

TRAIN

698,901

interactions

24,961 active users

For model fitting

VALIDATION

7,023

held-out positives

1 per user

For hyperparameter tuning

TEST

10,393

held-out positives

1 per user, novel recipes

For final evaluation

Time range 2000–2018 · 88.6% of ratings are 4+ stars · 100% nutrition coverage · 8,023 canonical ingredients

Dataset: Majumder et al., EMNLP 2019 · kaggle.com/datasets/shuyangli94/food-com-recipes-and-user-interactions

EDA of Interaction Train Dataset

Real user-recipe interactions at scale. No scraping required.

Sparsity

99.98%

User reviews

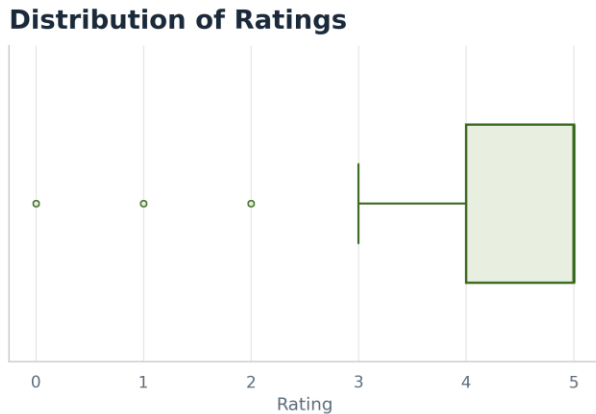
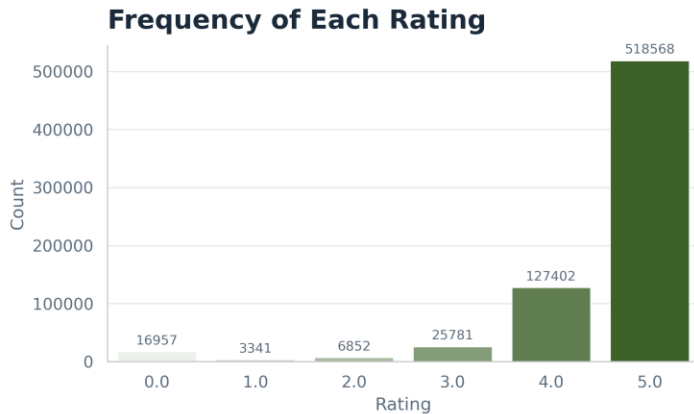
698,901

Unique Recipes

160,901

All users

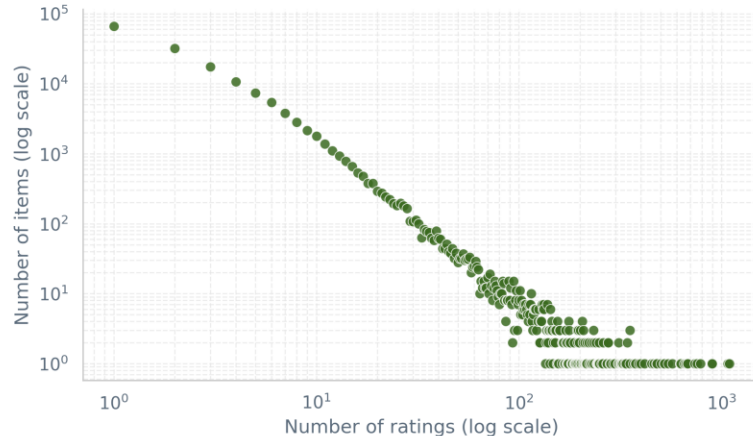
25,076



Long tail of interactions



Log-log popularity distribution



- The most popular item has ~1,000 ratings; counts collapse to near-zero within the first sliver of items.
- Tens of thousands of items have just a single rating — singletons dominate.
- The log-log line is roughly straight, the signature of a power-law (Zipfian) distribution.
- The tail is long but thin: high-rating items exist, but only a handful each.

Content Features of Recipe Dataset

Pantry

items that are more commonly found in pantry, popularity of some ingredients

- ingredients
- n_ingredients

Nutrition

*Avg ~450 cal, high sugar, moderate fat, low protein
A sweet tooth profile*

- nutrition

Diet

*User only consumes dishes with only vegetables as ingredients
A vegetarian profile*

- ingredients
- tags

Taste

User leans towards desserts & baking and "easy" tags reoccur

- Tags
- Name
- description

Two-stage pipeline

Recsys learning in Stage 1; constraint reasoning in Stage 2.

USER CONTEXT

Rating history · pantry · macro targets · dietary restrictions · α -weights



STAGE 1 · CANDIDATE GENERATION (REAL RECSYS)

MF, EASE, BPR, content TF-IDF, Sentence-BERT, hybrid, two-tower neural, sequential — learns taste from observed ratings. Returns top-200 candidates.



STAGE 2 · MULTI-CONSTRAINT RERANKING (X-FACTOR)

Scores each candidate on taste $\times$ pantry $\times$ nutrition $\times$ diet. Combines via α -weighting. Returns top-10.



Top-10 recommendations

Two complementary paradigms

Real recipe apps face two scenarios. Each needs a different family of models.

COLLABORATIVE FILTERING

Learns from rating co-occurrence — who rated what, finds taste-similar users.

ANSWERS

“What do users like me like?”

→ Wins on **WARM-ITEM** recommendation

FAILS WHEN

The recipe has no rater history — zero signal to extrapolate from.

CONTENT-BASED

Learns from recipe content directly — ingredients, tags, nutrition, name, description.

ANSWERS

“Who'd like this new recipe?”

→ Wins on **COLD-ITEM** recommendation

FAILS WHEN

Content is generic; doesn't capture nuanced user-specific taste.

HYBRID — BEST OF BOTH Combines CF taste signal with content features. The only family that handles **both** warm and cold scenarios.

The four constraint scores

Where the X-factor lives.

- s_taste** — predicted relevance from Stage 1 (continuous, [0,1])
- s_pantry** — non-staple ingredient overlap, soft ranking signal (continuous, [0,1])
- s_nutrition** — Gaussian proximity to macro targets (continuous, [0,1])
- s_diet** — hard filter: 1 if all restrictions met, 0 otherwise

$$\text{final}(u, r) = s_{\text{diet}} \times (\alpha_t \cdot s_{\text{taste}} + \alpha_p \cdot s_{\text{pantry}} + \alpha_n \cdot s_{\text{nutrition}})$$

Only s_{diet} is a hard filter; the other three are continuous ranking signals — weighted, never excluding. For dietary restrictions without tag coverage (keto, paleo, whole30, lactose-free, halal, low-sugar), s_{diet} is derived from macros plus an ingredient blacklist.

RESEARCH OBJECT

The $(\alpha_t, \alpha_p, \alpha_n)$ simplex. Where on this surface do recommendations land, and what does the optimal balance look like?

The α -sweep study

The trade-off curve is the headline empirical result.

We sweep α across the simplex and measure how each metric shifts.

HYPOTHESIS 1 single-objective optimization produces worse Useful Recall@K than balanced ranking

HYPOTHESIS 2 optimal α -weighting differs by user type — fitness-focused users weight nutrition higher; budget cooks weight pantry higher

HYPOTHESIS 3 as constraints tighten, learned taste signal remains material — distinguishing this from pure filtering

HEADLINE PLOT Useful Recall@K vs α -position. Inflection point visible. Per-persona overlays.

Runs on both evaluation tracks — warm-item LOO and cold-item — so we can study how the trade-off shifts across scenarios.

Two evaluation regimes, same architecture

Real recipe apps face both scenarios — we evaluate on both.

TRACK A · WARM-ITEM (standard CF)

TEST SET	~24K positives, time-based LOO on train
SIGNAL	Items have median 20 raters in train
QUESTION	What's the next item from the known library?
MODELS	CF Based Models
FLOOR	Popularity Recall@10 $\approx$ 3.0%
ANALOGY	“Trending recipes among users like me”

TRACK B · COLD-ITEM (content-aware)

TEST SET	10,393 positives, authors' test split
SIGNAL	Items have 0 raters in train (cold by design)
QUESTION	What novel recipe, before anyone has rated it?
MODELS	Content-aware only: TF-IDF, BERT, hybrid, 2-tower
FLOOR	0.0% — popularity has no signal for unseen
ANALOGY	“Surface new recipes before they go viral”

WHY BOTH Different models win different regimes. CF-only dominates warm; content-aware dominates cold. Only the hybrid handles both — the architectural payoff. The Stage 2 reranker and α -sweep run on both tracks.

Recipe apps need both. PantryPlate evaluates on both.

Three categories of metric

Each plays a distinct role. Together they evaluate the full system.

Predictive

"did the model guess right?"

- Recall@5, @10
- NDCG@10
- MRR

Mechanical

"does output have desired properties?"

- Pantry Hit Rate@K
- Nutrition Adherence@K
- Diet Compliance@K

Joint (signature)

"both at once"

- Useful Recall@K
- Multi-Constraint Score
- α -sweep curves

Bootstrap 95% CIs on all means. Wilcoxon signed-rank for model-vs-model comparisons.

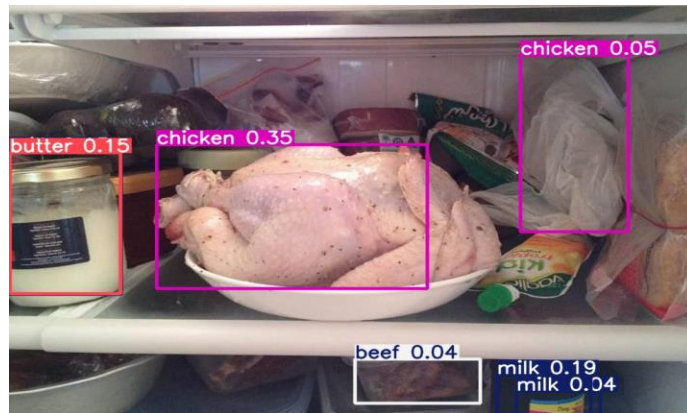
X-factor 1 (Computer vision)

Making it from a pure recommender problem ->
computer vision + recommender problem

Manually inputting the ingredients AND expiry date is:

- Time consuming
- Prone to human error

Due to time constraint,
we will just focus on ingredients identification



X-factor 2 (Shelf life)

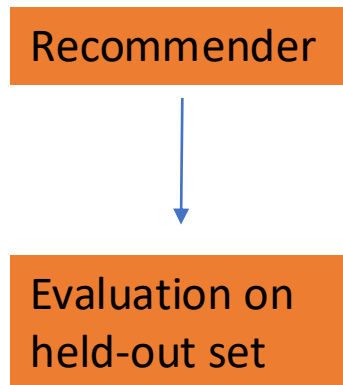
- 20-30% wasted ingredients per year due to expiry
- Synthetic expiry model by assigning shelf-life estimates to ingredients based on food category
(Perishable goods more likely to be recommended first)



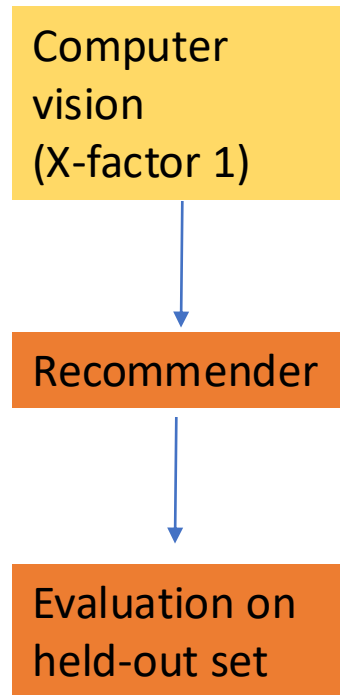
Shortest (1 week)

Longest (5 years)

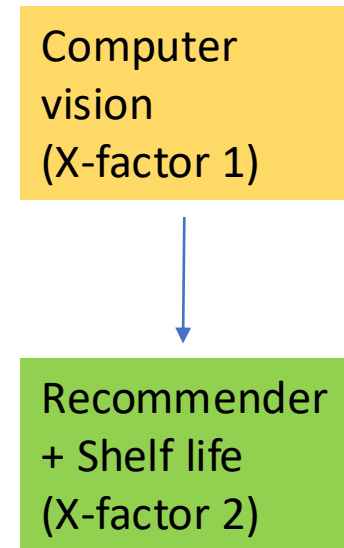
The original problem



The original problem + CV



Real-use case



How PantryPlate evolves from baseline to stretch

The recsys core stays constant; reach goals extend inputs and outputs.

